Document number: P4005R0
Audience: EWG

Ville Voutilainen
2026-02-02

A proposal for guaranteed-(quick-)enforced contracts

Abstract

This paper proposes a particular form of guaranteed-enforced contracts.

It follows the design guidance suggestions of P3919R0, and differs from P3911 by

- proposing a new keyword and new context-sensitive keywords that are separate and different from the P2900 ones
- proposing a simple approach using the existing expression evaluation semantics, which has the usual consequences, including but not limited to the ability to optimize based on the guaranteed-enforced assertions.

The main motivation

The main motivation is to provide a *guaranteed* assertion facility. Such a guarantee allows programs and users to be certain that the defined conditions are met, and such a guarantee therefore provides various forms of (mostly memory- and ub-) safety for callers, callees, readers, reviewers, and tools that they use.

There is a noticeably strong emphasis on the word *guarantee*.

That emphasis gives us a rationale for being able to declare such guaranteed assertions also on the function declaration level. Suggestions that such a facility is less necessary and enough functionality can be provided by just a guaranteed-assertion statement miss this goal. A guaranteed-assertion statement may or may not be present in a function body, but that's no guarantee at all that certain conditions must be met for a function to be entered, or that certain conditions must be met when a function returns normally. This would become rather problematic for opaque functions for which a definition isn't visible, either to human readers or tools.

The main parts

The proposal is as follows:

1. add an `entry_cond` for function declarations that specifies a mandatory entry condition that must evaluate to true to enter the function body, or otherwise call a violation handler and contract-terminate calls to said function, or directly contract-terminate calls to said function.
2. add a `return_cond` for function declarations that specifies a mandatory condition that must evaluate to true when the function returns normally, or otherwise call a violation handler and contract-terminate, or directly contract-terminate the program when the function returns.
3. add a `mandatory_assert` statement that specifies a condition that must evaluate to true when said statement is evaluated, or otherwise call a violation handler and contract-terminate, or directly contract-

terminate the program.

In other words,

```
void f(int x) entry_cond(x >= 0);
void use_it()
{
  f(-42); // entry_cond not met, will not continue to the subsequent code
  void(*p)(int) = f;
  p(-666); // entry_cond not met, will not continue to the subsequent code
}

int g(int x) return_cond(r: r >= 0)
{
  if (x == 42)
    return 666; // OK
  else
    return -1; // return_cond not met, will not continue to the subsequent code
  if (x >= 0)
    return x; // OK
  else
    return x; // return_cond not met, will not continue to the subsequent code
}

void h(int x)
{
  mandatory_assert(x >= 0); // will not continue to the subsequent code if x is negative
}
```

The pseudo-specification (no formal wording yet) for a call of a function that declares an entry_cond is that after parameters have been initialized and before the function body is entered, the expression(s) in the entry_cond(s) are evaluated (as if inside the body), and if any such expression results in false, a violation handler is called and then the program is contract-terminated, or the program is directly contract-terminated.

Usual expression evaluation rules apply. The expression is evaluated only once, unless it can be proven not to have side effects. The expression doesn't have to be evaluated at all if its result can be otherwise determined and there are no side effects.

The pseudo-specifications for return_cond and mandatory_assert are similar. A return_cond is evaluated when the function exits via a normal return, before the control returns to the caller.

Additional semantic bits, and differences from P2900 and P3911

- a condition of an entry_cond is evaluated whenever a function is called, and is then evaluated either with the 'enforce' or the 'quick_enforce' semantic. The evaluation semantic is never 'ignore' nor 'observe'. Not Even for Indirect Calls.
- the conditions of return_conds and guaranteed_asserts are likewise evaluated with the 'enforce' or the 'quick_enforce' semantic, and never with 'ignore' nor 'observe'.
- the selection mechanism for which evaluation semantic is used for any such guaranteed-enforced assertion is implementation-defined, but a conforming implementation must implement both the option of evaluating such a guaranteed-enforced assertion with the 'enforce' semantic and the option of evaluating such a guaranteed-enforced assertion with the 'quick_enforce' semantic.
- the evaluation of such assertions does not translate exceptions arising out of the evaluation of the condition in any way. Such exceptions fly out of the assertion, but like in P2900, cannot fly through noexcept barriers of functions.
- the evaluation of such assertions follows the usual expression evaluation rules. There is no 'constification' or other additional transform of what is evaluated.

- for any two function declarations, the declarations are ODR-different if they have different sets of entry_conds or return_conds.

What is proposed here is that it is ill-formed to mix guaranteed-enforced assertions and P2900 assertions in the same function declaration. That is the most future-compatible approach given the C++26 time frame.

While not really a semantic bit, and instead QoI, it's expected (but not required) that the ODR-difference is ensured by implementations by mangling the entry_cond/return_cond into the mangled function name.

This mangling makes it safe to enable compiler optimizations based on knowledge of the results of guaranteed-enforced assertions. If an assertion would be changed so that the optimizations are no longer valid, the program will not link any more due to a visible ABI break caused by a name mangling change.

Implementation experience

The full form of this proposal hasn't been implemented yet.

All the constituent bits have been:

1. we have implementation experience with always-enforced contract assertions
2. we have implementation experience with a mix of contract assertions with 'constification' and without
3. we have implementation experience with a mix of contract assertions that translate predicate exceptions to contract violations and assertions that don't
4. we have implementation experience with mangling (almost?) arbitrary expressions in function declarations